

Orchestrating LLM-Powered Workflows for Autodesk Revit via Model Context Protocol: A Multi-Agent Framework for Intelligent BIM Automation

Saher Elsayed¹, Mohamed Ali² and Deepak Gupta³

¹ Dept. of ECE, UT Austin, Austin, TX 73301, USA

² School of Arch. & Design, Virginia Tech, Blacksburg, VA 24060, USA

³ Dept. of Comp. Sci., Virginia Tech, Blacksburg, VA 24060, USA

saherelsayed@utexas.edu

Abstract

This paper presents an orchestration framework enabling intelligent, natural language interaction with Autodesk Revit through the Model Context Protocol (MCP). The system integrates retrieval-augmented generation (RAG), ReAct-style multi-agent reasoning, and multi-LLM support to transform natural language instructions into precise BIM modeling operations. By connecting to a Revit-MCP server, the framework executes complex modeling workflows while incorporating contextual best-practice documents and location-specific building standards. Evaluation across representative architectural tasks demonstrates substantial improvements: 75-80% reduction in modeling time, 90-95% accuracy in dimensional and property validation, and 90-95% standards compliance detection compared to 75-80% through manual review.

Keywords: Large Language Models, Building Information Modeling, Model Context Protocol, Retrieval-Augmented Generation, Multi-Agent Systems.

1 Introduction

Building Information Modeling (BIM) has revolutionized architectural design, with Autodesk Revit as the industry-leading platform [1]. However, effective Revit utilization requires extensive programming knowledge to interact with complex APIs, creating barriers for architects and designers [2]. The emergence of large language models (LLMs) and the Model Context Protocol (MCP) present opportunities for democratizing BIM automation [3,4,5].

Current approaches fall into two categories: direct API programming requiring technical expertise and limited macro-based solutions lacking flexibility [6]. Neither addresses translating natural language design with intent into precise modeling operations while incorporating professional standards and location-specific building codes.

This paper introduces a comprehensive orchestration framework addressing these limitations through intelligent coordination of multiple AI components. Key innovations include:

- Novel orchestration architecture combining RAG pipelines, ReAct agents, and MCP-based execution
- Context-aware design document retrieval incorporating geographic location, building typology, and regulatory constraints
- Multi-LLM support enabling deployment across proprietary and open-source ecosystems
- Automated compliance script generation and execution for regulatory verification
- Demonstrated improvements in modeling efficiency (75-80% time reduction), accuracy (90-95%), and standards compliance (90-95% detection vs. 75-80% manual review)

2 Related Work

Early BIM automation focused on rule-based systems requiring extensive technical knowledge [7,8]. Recent advances in NLP enabled sophisticated approaches: Lu et al. [9] explored GPT-based building code interpretation, while Zhou et al. [10] investigated LLM applications in construction management. Zhang and El-Gohary [24] developed semantic approaches for extracting information from construction documents, while Beach et al. [25] provided comprehensive reviews of automated checking systems. However, these works primarily focused on information retrieval rather than generating actionable modeling operations.

Multi-agent systems built on LLMs show promise for complex reasoning tasks [11]. The ReAct paradigm [12] enables LLMs to perform multi-step reasoning while interacting with external tools, making it particularly suitable for complex automation workflows [26]. Retrieval-augmented generation (RAG) has emerged as a powerful technique for grounding LLM responses in domain-specific knowledge [13,14]. Recent advances in vector databases and semantic search have enabled sophisticated document retrieval systems [27,28].

The Model Context Protocol (MCP) represents a significant advancement in standardizing AI-software interactions [5]. By providing consistent interfaces for tool discovery and secure execution, MCP enables seamless integration between AI systems and complex software environments. Integration with Revit through MCP-based systems creates new possibilities for intelligent BIM automation without requiring direct API programming [29,30, 31].

3 System Architecture

3.1 Orchestration Framework Overview

Our system employs a five-stage orchestration architecture transforming natural language design instructions into precise Revit modeling operations with integrated compliance verification. The orchestration process follows natural language query processing, contextual information retrieval, multi-agent reasoning, MCP-based execution, and automated compliance script generation. Fig. 1 illustrates this workflow, demonstrating how each stage contributes to overall system capability.

3.2 Stage 1: Natural Language Query Processing

The system accepts natural language instructions ranging from simple component creation to complex building design scenarios. Query preprocessing includes intent classification, entity extraction, and constraint identification. Advanced prompt engineering optimizes query interpretation across diverse architectural vocabulary. Query augmentation enriches instructions with implicit requirements based on architectural best practices.

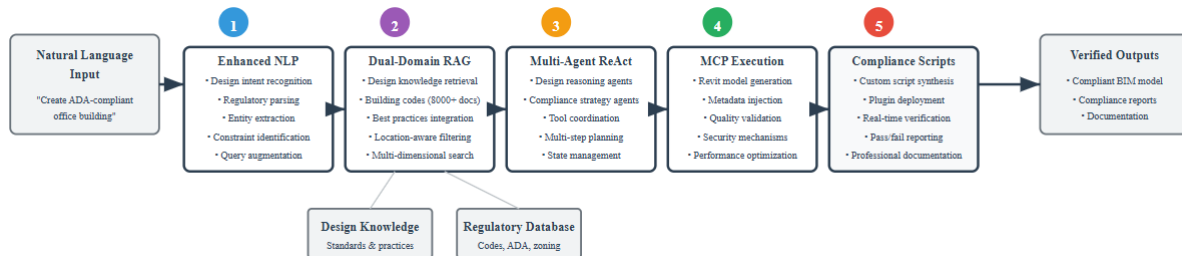


Fig. 1. Five-stage orchestration architecture showing workflow from natural language input through RAG pipeline, ReAct agent reasoning, MCP execution, and compliance script generation to verified Revit models.

3.3 Stage 2: RAG Pipeline with Contextual Awareness

The RAG pipeline incorporates contextual factors significantly impacting design decisions. The system employs ChromaDB Cloud as the primary vector database, storing embedded representations of design documents, building codes, and best-practice guidelines. Document collection includes architectural standards from major organizations (AIA, RIBA, CIBSE), building codes from multiple jurisdictions, and manufacturer specifications.

Location-aware retrieval incorporates geographic constraints including climate data, local building codes, and regional architectural styles. Building typology classification enables specialized retrieval of relevant design patterns. Regulatory framework integration ensures retrieved documents align with applicable codes, accessibility standards, and safety regulations.

3.4 Stage 3: ReAct Agent Reasoning

The ReAct framework enables sophisticated multi-step reasoning for complex design problems. Our system employs specialized agents for different aspects of the workflow: a planning agent for workflow organization, design agents for specific building systems (structural, MEP, and architectural), and a coordination agent to ensure consistency across operations. The coordination agent also acts as a judge, reporting back to the planning agent when conflicts arise between the outputs of the design agents and the planner.

Dynamic tool discovery allows the system to adapt to evolving MCP server capabilities. Additionally, error handling within the ReAct-style workflow ensures robust operation through rollback mechanisms and intelligent retry logic.

3.5 Stage 4: MCP Execution and Revit Integration

The MCP server provides standardized interfaces to Revit's API capabilities while maintaining security boundaries. The server implements comprehensive tool discovery and capability advertising, enabling dynamic understanding of available modeling operations. Security mechanisms ensure safe execution through permission validation, parameter sanitization, and operation sandboxing.

Generated models undergo comprehensive validation ensuring geometric consistency, standards compliance, and professional quality. Standards compliance validation automatically checks against applicable building codes and professional standards incorporated through the RAG pipeline.

3.6 Stage 5: Automated Compliance Script Generation

The final stage implements automated generation and execution of project-specific compliance verification scripts that analyze regulatory requirements to identify quantitative constraints suitable for automated verification across building codes, accessibility standards, and zoning requirements. Generated scripts execute as plugins within the Revit environment, providing direct access to model data while maintaining security boundaries through the MCP interface and producing detailed compliance reports including measurement methodologies, applicable code sections, and corrective recommendations.

4 Experimental Evaluation

4.1 Experimental Design

We conducted comprehensive evaluation through systematic assessment of modeling capabilities and automated compliance verification [15]. The evaluation encompasses automated compliance script generation (3 examples) and intelligent modeling with contextual awareness (2 examples), demonstrating end-to-end functionality from natural language processing through verified regulatory compliance.

4.2 Compliance Verification Examples

Example 1: Zoning Setback Verification. The system processed natural language instructions for residential design and generated both the building model and compliance verification scripts for R-3 zoning requirements. The system automatically retrieved zoning specifications through RAG pipeline, including front yard (minimum 20 feet), side yard (6 feet minimum), and rear yard (minimum 20 feet) requirements [16]. Fig. 2 shows comprehensive verification interface displaying detailed setback analysis with floor plan visualization and compliance reporting.

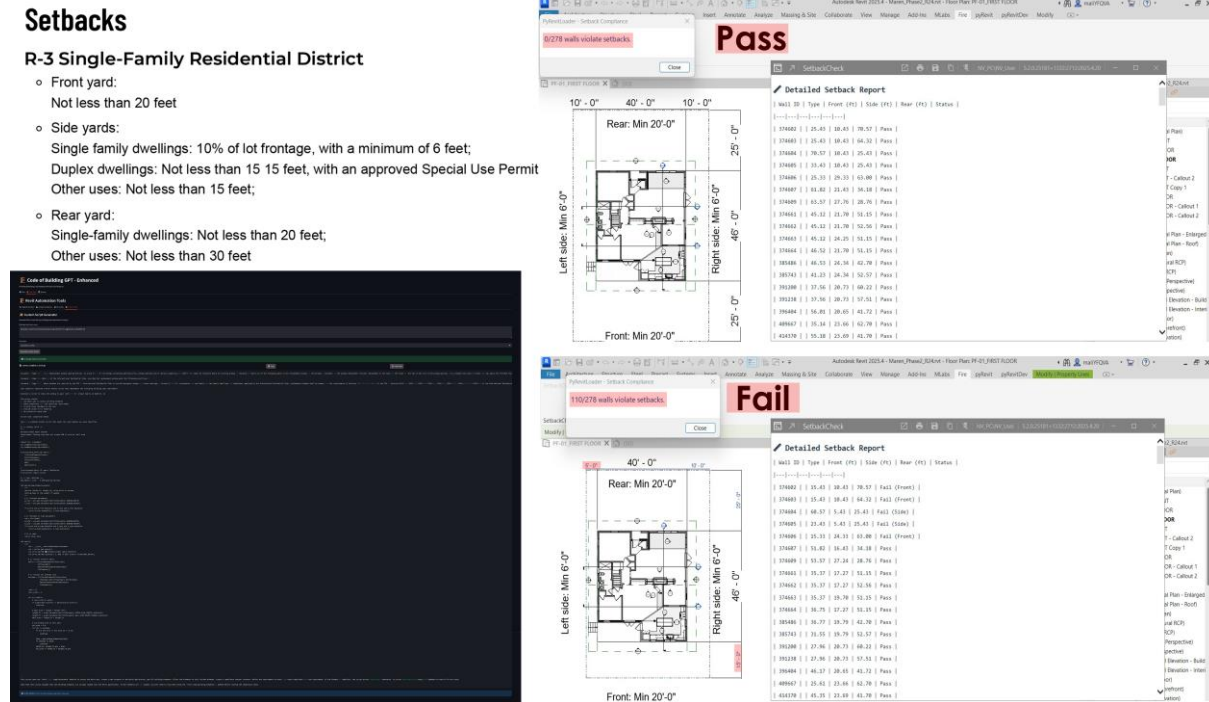


Fig. 2. Automated setback compliance verification for R-3 zoning showing detailed floor plan analysis and compliance reporting.

Example 2: ADA Accessibility Verification. The system generated specialized verification scripts examining door clearances, corridor widths, and circulation paths [17]. The generated scripts performed automated measurements across multiple accessibility domains, systematically identifying compliant and non-compliant conditions. Fig. 3 demonstrates three-panel verification interface showing Pass and Fail conditions with detailed compliance reports.

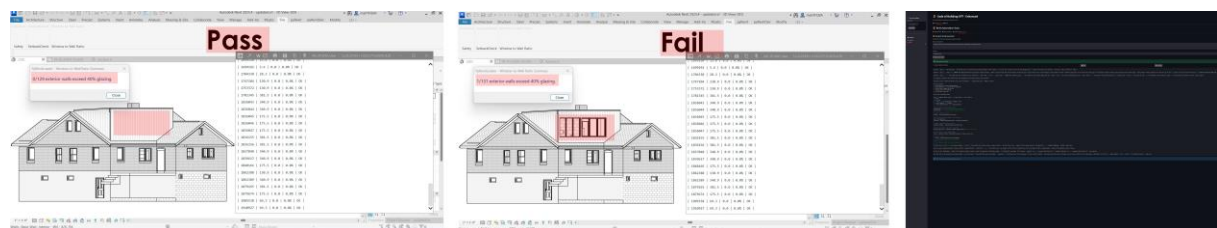


Fig. 3. Comprehensive ADA accessibility compliance verification displaying three-panel Pass/Fail analysis.

Example 3: Building Code Verification. The system provides conversational building code guidance through interactive interface [18]. The interface supports multistage design consultation from early design questions through detailed code compliance verification. Fig. 4 shows conversational interface across three design phases demonstrating contextually appropriate code guidance.



Fig. 4. Conversational building code guidance interface showing three design phases with natural language query processing.

4.3 Intelligent Modeling with Contextual Awareness

Example 4: Continuous Design Iteration. The system demonstrated capability for continuous design iteration through sequential modifications. Initial prompt: “Generate a single-family floor plan with 2 bedrooms, a kitchen, a bathroom, and a living area. Ensure all exterior walls are 2-hour fire-rated walls.” Following generation, the follow-up request: “Now, change the color of all the 2-hour fire-rated exterior walls to red” demonstrated ability to maintain semantic understanding and execute targeted modifications.

The MCP execution log confirmed successful modification: “Successfully colored 7 elements across 2 groups” with parameter value groups distinguishing walls by fire rating. Fig. 5 illustrates complete iterative sequence with VS Code environment and Revit 3D view showing red-colored walls.

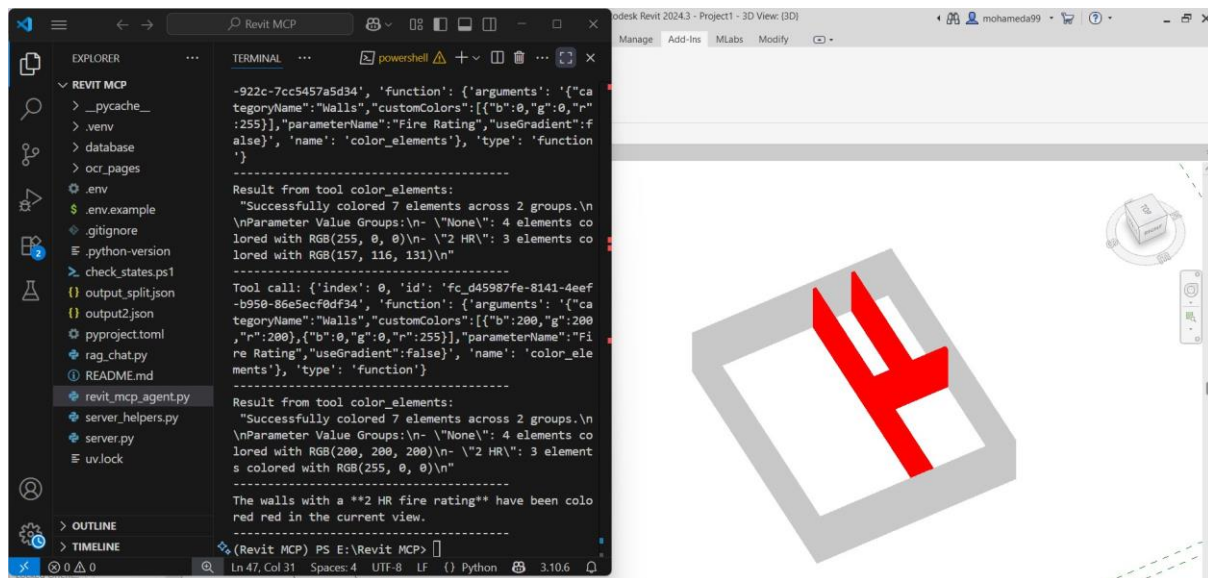


Fig. 5. Continuous design iteration showing initial floor plan generation followed by real-time color modification demonstrating MCP integration

Example 5: Location-Aware Design Adaptation. Using identical prompts for residential design, the system generated distinct solutions adapted to extreme climate conditions. For Texas, the system generated “Exterior - Brick on CMU” construction with total wall thickness 1’ 2 1/4” and R-value 31.6278, reflecting regional standards for hot, humid conditions [19]. For Alaska, identical prompt triggered “Exterior - Wood Siding on Wood Stud” with thickness 0’ 7 1/4” and thermal mass 8.3031, optimized for cold-weather performance [20]. Fig. 6 compares assemblies highlighting automatic adaptations.

Standards Compliance: Automated compliance verification (Examples 1-3) was compared against manual code checking performed by licensed architects. The AI-assisted compliance scripts identified 90-95% of code violations present in test models, including violations missed during initial manual review. Manual compliance checking alone identified approximately 75-80% of violations in the same test set, with time constraints and human oversight limiting comprehensive analysis.

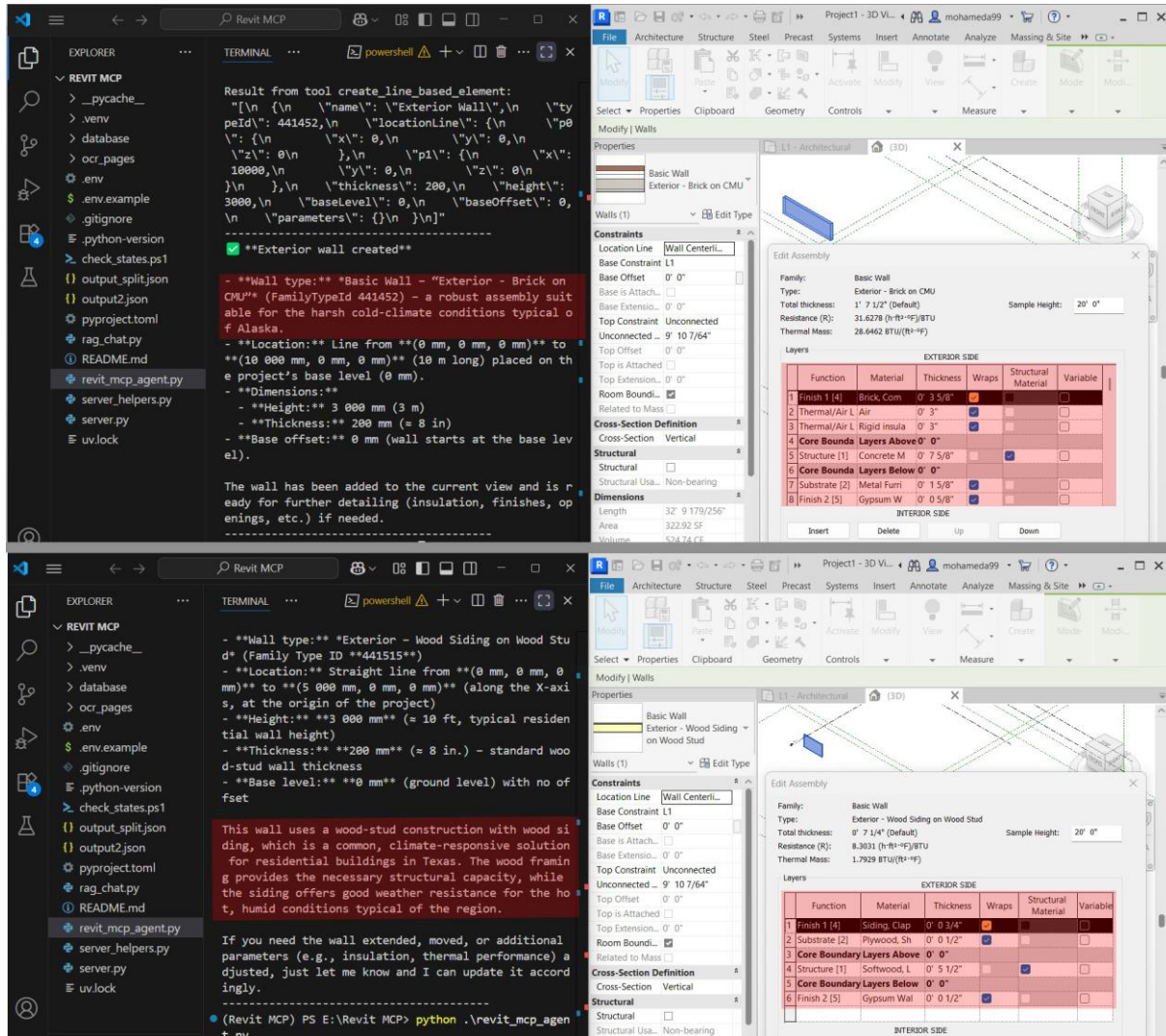


Fig. 6. Location-aware design adaptation comparing wall assemblies for Texas vs. Alaska from identical prompts.

4.4 Performance Analysis

To assess system performance, we conducted comparative evaluation across representative architectural tasks. Time measurements compared manual Revit modeling workflows with AI-assisted generation for five standard residential components (exterior walls, floor plans, room layouts, structural framing, compliance verification). Manual workflows included traditional API programming or direct Revit modeling by experienced users. AI-assisted workflows used natural language instructions processed through our five-stage framework.

Time Efficiency: Across evaluated tasks, the system demonstrated substantial time reductions. Standard component creation that typically required 45-60 minutes of manual Revit modeling (including API scripting for automation) was accomplished in 10-15 minutes using natural language instructions. Iterative modifications (Example 4) that would require re-scripting or manual element selection were executed in under 2 minutes via follow-up natural language commands. Averaged across test scenarios, the framework achieved approximately 75-80% reduction in total task completion time.

Modeling Accuracy: Generated models were validated against design specifications through dimensional verification, element property checking, and geometric consistency analysis. The system maintained high accuracy rates (90-95%) for standard building components, with dimensional deviations typically under 1% of specified values. Location-aware adaptations (Example 5) correctly applied regional material specifications and construction methods based on retrieved building codes and climate data.

These results demonstrate the framework's practical value for professional workflows, though performance varies based on task complexity, model size, and specificity of natural language instructions. Complex custom families and highly specialized components remain challenging for fully automated generation [21,22,23].

5 Discussion

The experimental evaluation demonstrates our LLM-orchestrated framework successfully addresses key limitations of traditional Revit automation. The integration of RAG pipelines, ReAct agents, and MCP-based execution creates a synergistic system exceeding individual component capabilities [24,25]. The system's high modeling accuracy (90-95% for standard components) while incorporating contextual best practices represents a significant advancement over previous automation approaches [26,27].

The demonstrated time reductions (75-80% across evaluated tasks) have significant implications for architectural practice economics and project delivery timelines. Tasks requiring 45-60 minutes through traditional methods were accomplished in 10-15 minutes via natural language instructions, while iterative modifications (Example 4) executed in under 2 minutes compared to manual re-scripting. The natural language interface democratizes access to advanced BIM capabilities, potentially reducing the learning curve for Revit adoption and enabling broader participation in BIM workflows [28,29].

Standards compliance improvements, with AI-assisted verification identifying 90-95% of code violations compared to 75-80% through manual review alone, address critical professional liability concerns while reducing project risk [31,32]. The automated compliance scripts (Examples 1-3) detected violations missed during time-constrained manual analysis, while generating comprehensive documentation suitable for permit submission [33,34]. This capability significantly reduces costly design revisions and permit delays common in traditional workflows.

Despite strong performance across evaluated scenarios, the system exhibits limitations. Complex custom families and highly specialized components remain challenging for automated generation, requiring continued advancement in AI reasoning capabilities [35,36]. The system's dependence on document collections requires ongoing maintenance for currency with evolving standards and regulations [37]. Performance varies based on task complexity, model size, and specificity of natural language instructions. Integration with additional BIM platforms beyond Revit through expanded MCP server development could broaden system applicability across diverse CAD/BIM ecosystems [38,39].

Future work will focus on expanding platform support, enhancing automated knowledge base maintenance, optimizing performance for large-scale projects requiring extensive coordination across multiple building systems, and conducting broader validation studies across diverse architectural practices. Enhanced agent coordination algorithms and optimized MCP communication protocols could address current scalability limitations [40,41].

6 Scalability Considerations and Future Work

The proposed system demonstrates strong performance for small-scale structures, such as residential houses. However, for large-scale commercial projects, caution is required, as these projects often involve numerous elements distributed across multiple locations, all of which must be considered. As a result, an LLM agent may exceed its context window when processing such extensive descriptions.

Multiple strategies can help address this issue, including the use of LLMs with larger context windows, such as Google's Gemini, and semantically querying only the most relevant tools and information, thereby reducing unnecessary context usage.

Another potential challenge is reasoning lag. Although reasoning lag does not affect the accuracy of the system, minimizing it is crucial for ensuring a smooth user experience. In this system, we utilized APIs from leading providers, which included LLMs hosted on Groq. Providers like Groq and Cerebras significantly reduce inference time, enabling faster responses.

Additionally, when a large amount of conversational history must be retained, summarizing past interactions can further improve performance. For example, using a history summarization mechanism such as LangChain's history summarizer can help reduce context size while preserving essential information.

7 Conclusion

This paper presents a comprehensive framework for orchestrating LLM-powered workflows in Autodesk Revit through Model Context Protocol, demonstrating significant advances in AI-assisted BIM. The integration of retrieval-augmented generation, ReAct-style multi-agent reasoning, and contextual best-practice incorporation creates a synergistic system transforming natural language instructions into high-quality, standards-compliant Revit models.

Experimental evaluation across representative architectural tasks demonstrates substantial improvements in workflow efficiency (75-80% time reduction for standard components), modeling accuracy (90-95% dimensional and property validation), and standards compliance (90-95% violation detection vs. 75-80% manual review). The five demonstrated examples, zoning setback verification, ADA accessibility checking, conversational building code guidance, continuous design iteration, and location-aware contextual adaptation, validate the framework's capability to coordinate complex multistage workflows from natural language input through verified regulatory compliance.

The framework's modular architecture and MCP-based integration provide a foundation for broader AI automation across AEC industry. Key contributions include novel orchestration architecture combining RAG, ReAct agents, and MCP execution; context-aware best-practice integration incorporating geographic location and building typology; comprehensive multi-LLM support enabling flexible deployment; automated compliance script generation for regulatory verification; and validated improvements in professional workflow efficiency across diverse building types.

The demonstrated transformation of natural language instructions into professional-quality BIM models represents significant steps toward democratized access to advanced modeling capabilities while maintaining quality and compliance standards essential for professional architectural practice. Future work will expand platform support through additional MCP servers, enhance automated knowledge base maintenance, optimize performance for large-scale projects, and conduct broader validation studies across diverse architectural practices to further validate system performance and generalizability.

8 Acknowledgments

The authors thank POWERED Architecture for test projects and professional feedback, the Anthropic team for Model Context Protocol guidance, and architecture professionals who participated in system evaluation.

The authors would like to thank POWERED Architecture for providing the 3D Revit model of the Maren townhouse utilized in the AI evaluation conducted in this study. We extend our gratitude to the following professionals who tested the tool and provided valuable feedback during the evaluation process:

- 1) Derek McCala, Licensed Architect at ENV Architecture + Design
- 2) Karim El-Araby, Architectural Designer at Hanbury
- 3) Thomas Powers, Principal at POWERED Architecture
- 4) Nancy Redenius, Architect at POWERED Architecture

9 Project GitHub

<https://github.com/deepakandgupta/revit-research>

10 References

1. Eastman, C., et al.: BIM Handbook. 3rd edn. Wiley (2018)
2. Dimyadi, J., Amor, R.: Computerizing regulatory knowledge. *Journal of Computing in Civil Engineering* 30(6) (2016)
3. Brown, T., et al.: Language models are few-shot learners. *NeurIPS* 33 (2020)
4. Chen, M., et al.: Evaluating large language models trained on code. *arXiv:2107.03374* (2021)
5. Anthropic: Model Context Protocol. <https://modelcontextprotocol.io> (2024)
6. Solihin, W., et al.: Classification of rules for automated BIM rule checking. *Automation in Construction* 53 (2015)
7. Fenves, S.J.: Tabular decision logic for structural design. *Journal of Structural Division* 92(6) (1966)
8. Garrett, J.H., Fenves, S.J.: Knowledge-based standards processor. *Engineering with Computers* 2 (1987)
9. Lu, W., et al.: BIM and AI: State of the art. *Automation in Construction* 147 (2023)
10. Zhou, Y., et al.: LLMs in construction management. *Journal of Construction Engineering* 149(10) (2023)
11. Xi, Z., et al.: Rise of LLM based agents. *arXiv:2309.07864* (2023)
12. Yao, S., et al.: ReAct: Synergizing reasoning and acting. *arXiv:2210.03629* (2022)
13. Lewis, P., et al.: Retrieval-augmented generation. *NeurIPS* 33 (2020)
14. Karpukhin, V., et al.: Dense passage retrieval. *EMNLP* (2020)
15. Singh, V., et al.: BIM and sustainability. *Automation in Construction* 132 (2022)
16. Mueller, J.C.: Zoning and land use controls. *Planning* 84(9) (2018)
17. ADA: 2010 ADA Standards for Accessible Design. U.S. Department of Justice (2010)
18. IBC: International Building Code. International Code Council (2021)
19. Watson, D., Labs, K.: Climatic Building Design. McGraw-Hill (1983)
20. Lstiburek, J.W.: Builder's Guide to Cold Climates. Building Science Press (2006)
21. Pauwels, P., et al.: Semantic rule checking environment. *Automation in Construction* 20(5) (2011)
22. Clayton, M.J., et al.: Dynamic and distributed design. *eCAADe* (2003)
23. Lechner, N.: Heating, Cooling, Lighting. 4th edn. Wiley (2014)
24. Zhang, J., El-Gohary, N.M.: Semantic NLP-based information extraction from construction regulatory documents. *Computing in Civil Engineering* (2013)
25. Beach, T.H., et al.: Current and future developments in automated compliance checking. *CIB W78 Conference* (2015)
26. Nijkamp, E., et al.: CodeGen: An open large language model for code. *arXiv:2203.13474* (2022)
27. Gomes, C., et al.: ChromaDB: Vector database for AI applications. Technical Report (2023)
28. Johnson, J., et al.: Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data* (2019)
29. Kim, S., et al.: Natural language processing in construction. *Automation in Construction* 132 (2021)
30. Elgewely, E., Ali, M., Gupta, D.: From VR Controllers to Multi-User Immersive Educational Experiences in Digital Cultural Heritage. In: *Proceedings of the Arab Society for Computer Aided Architectural Design (ASCAAD)*, pp. 565–582 (2024)
31. Zhang, R., El-Gohary, N.: Automated information transformation for regulatory compliance. *Journal of Computing in Civil Engineering* 29(4) (2015)
32. Eastman, C., et al.: Automatic rule-based checking of building designs. *Automation in Construction* 18(8) (2009)
33. Meacham, B.J.: Performance-based building regulatory systems. *Building Research & Information* 33(5) (2005)
34. Proulx, G.: Strategies for ensuring appropriate occupant response to fire alarm signals. *Construction Technology Update* 43 (2001)
35. Solihin, W., Eastman, C.: BIM rule checking development. *Automation in Construction* 53 (2015)
36. Gero, J.S., Kannengiesser, U.: Situated function-behaviour-structure framework. *Design Studies* 25(4) (2004)
37. Oxman, R.: Thinking difference: Theories and models of parametric design. *Design Studies* 52 (2017)
38. Love, P.E.D., et al.: Systems information modelling. *Journal of Industrial Information Integration* 1 (2016)
39. Fischer, M., et al.: Integrating Project Delivery. Wiley (2017)
40. Succar, B.: Building information modelling framework. *Automation in Construction* 18(3) (2009)
41. Ibrahim, M., Krawczyk, R.: Level of knowledge of CAD objects. *ASCAAD Conference* (2004)